2019 IEEE 30th International Conference on Application-Specific Systems, Architectures and Processors (ASAP)

PPAC: A Versatile In-Memory Accelerator for Matrix-Vector-Product-Like Operations

(Invited Paper)

Oscar Castañeda, Maria Bobbett, Alexandra Gallyas-Sanhueza, and Christoph Studer

School of Electrical and Computer Engineering, Cornell University, Ithaca, NY
e-mail: {oc66, mb2567, ag753, studer}@cornell.edu; website: http://vip.ece.cornell.edu

***Abstract*—Processing in memory (PIM) moves computation into memories with the goal of improving throughput and energy-efficiency compared to traditional von Neumann-based architectures. Most existing PIM architectures are either general-purpose but only support atomistic operations, or are specialized to accelerate a single task. We propose the Parallel Processor in Associative Content-addressable memory (PPAC), a novel in-memory accelerator that supports a range of matrix-vector-product (MVP)-like operations that find use in traditional and emerging applications. PPAC is, for example, able to accelerate low-precision neural networks, exact/approximate hash lookups, cryptography, and forward error correction. The fully-digital nature of PPAC enables its implementation with standard-cell-based CMOS, which facilitates automated design and portability among technology nodes. To demonstrate the efficacy of PPAC, we provide post-layout implementation results in 28nm CMOS for different array sizes. A comparison with recent digital and mixed-signal PIM accelerators reveals that PPAC is competitive in terms of throughput and energy-efficiency, while accelerating a wide range of applications and simplifying development.**

I. Introduction

Traditional von Neumann-based architectures have taken a variety of forms that trade-off flexibility with hardware efficiency. Central processing units (CPUs) are able to compute any given task that can be expressed as a computer program. In contrast, application-specific integrated circuits (ASICs) are specialized to accelerate a single task but achieve (often significantly) higher throughputs and superior energy-efficiency. In between reside graphics processing units (GPUs) and field-programmable gate arrays (FPGAs), that are more specialized than CPUs, but typically offer higher throughput and energy-efficiency for the supported tasks. The ever-growing gap between computing performance and memory access times has lead today's von Neumann-based computing systems to hit a so-called "memory wall" [1], which describes the phenomenon that most of a system's bandwidth, energy, and time is consumed by memory operations. This problem is further aggravated with the rise of applications, such as machine learning, data mining, or 5G wireless systems, where massive amounts of data need to be processed at high rates and in an energy-efficient way.

The work of OC, AGS, and CS was supported by ComSenTer, one of the JUMP centers sponsored by the semiconductor research corporation (SRC), and by SRC nCORE task 2758.004 and the US National Science Foundation (NSF) grant ECCS-1740286 under the E2CDA program. The work of MB was supported by the Cornell University Engineering Learning Initiatives (ELI).

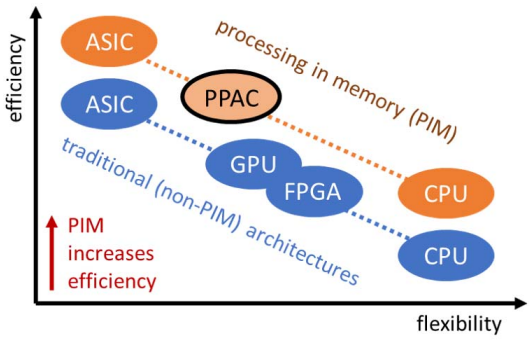


Fig. 1. Idealized efficiency-flexibility trade-off for different hardware architectures. Processing in memory (PIM) aims at increasing throughput and energy-efficiency by moving computation into memories. The proposed Parallel Processor in Associative CAM (PPAC) is a fully-digital in-memory accelerator that supports a range of matrix-vector-product-like operations.

A. Processing In Memory

Processing in memory (PIM) is an emerging computing paradigm that promises to tear down the memory wall [2]. Put simply, PIM brings computation closer to the memories, with the objective of reducing the time and energy of memory accesses, which ultimately increases the circuit's overall efficiency (see Fig. 1 for an illustration). The application of PIM to general-purpose processors has been explored recently in [3]–[5]. While such PIM-aided CPUs enable improved throughput and energy-efficiency for certain memory-intensive workloads, the supported PIM operations are typically limited to atomistic operations (such as bit-wise AND/NOR). As a consequence, executing even slightly more complex operations (such as multi-bit additions or multiplications) requires a repeated use of the supported PIM operations; this prevents such architectures from reaching the throughput and energy-efficiency required in many of today's applications. Hence, a number of PIM-based ASICs have been explored recently in [6]–[10]. Such solutions generally excel in throughput and energy-efficiency, but have limited applicability, often accelerating a single task only. For example, the PIM-ASIC in [6] is designed to accelerate neural network inference using mixed-signal techniques, but suffers from effects caused by noise and process variation; this prevents its use in applications in which the least significant bit must

2160-052X/19/$31.00 ©2019 IEEE
DOI 10.1109/ASAP.2019.000-9

be computed accurately (e.g., in cryptography, forward error correction, or locality-sensitive hashing).

B. Contributions

While a range of PIM-based ASICs and CPUs have been proposed in recent years, to the best of our knowledge, no PIM-based solutions exist that simultaneously offer high flexibility and high efficiency. To fill in this void in the trade-off space with PIM-based hardware solutions (see Fig. 1), we propose a novel, versatile in-memory processor called *Parallel Processor in Associative Content-addressable memory (PPAC)*, which supports a range of matrix-vector-product (MVP)-like operations. PPAC is designed entirely in digital standard-cell-based CMOS, accelerates some of the key operations in a wide range of traditional and emerging applications, and achieves high throughput and energy-efficiency for the supported tasks. The proposed architecture consists of a two-dimensional array of latch-based bit-cells that support two types of binary-valued operations; each row of the PPAC array is equipped with a row arithmetic-logic unit (ALU) that supports a variety of tasks, including content-addressable memory (CAM) functionality, Hamming-distance calculation, one- and multi-bit MVPs, Galois field of two elements GF(2) MVPs, and programmable logic array (PLA) functionality. We provide post-layout implementation results in a 28 nm CMOS technology and compare the area, throughput, and energy-efficiency to that of recent related accelerators.

C. Paper Outline

The rest of the paper is organized as follows. In Section II, we describe the operating principle and architecture of PPAC. In Section III, we detail all operation modes and outline potential use cases. In Section IV, we present post-layout implementation results and compare PPAC to related accelerator designs. We conclude in Section V.

II. PPAC: Parallel Processor in CAM

We now describe the operating principle of PPAC and introduce its architecture. In what follows, the terms "word" and "vector" will be used interchangeably—an N-bit word can also be interpreted as a binary-valued vector of dimension N.

A. Operating Principle

PPAC builds upon CAMs, which are memory arrays that compare all of their M stored N-bit words $\mathbf{a}_m$, $m = 1, \ldots, M$, with an N-bit input word $\mathbf{x}$ to determine the set of stored words that match the input. Conceptually, the functionality of a CAM can be described as a memory in which every bit-cell contains an XNOR gate to determine whether the stored value $a_{m,n}$ matches the input bit x_n, $n = 1, \ldots, N$. A match is then declared only if all the N bits in $\mathbf{a}_m$ match with the N bits of the input $\mathbf{x}$. Mathematically, the functionality of a CAM can be expressed in terms of the *Hamming distance* $h(\mathbf{a}_m, \mathbf{x})$, which indicates the number of bits in which $\mathbf{a}_m$ and $\mathbf{x}$ differ. A CAM declares a match between the stored word $\mathbf{a}_m$ and the input word $\mathbf{x}$ if $h(\mathbf{a}_m, \mathbf{x}) = 0$. As it will become useful later, one can alternatively describe a CAM's functionality using the *Hamming similarity*, which we define as $\overline{h}(\mathbf{a}_m, \mathbf{x}) = N - h(\mathbf{a}_m, \mathbf{x})$, and corresponds to the number of bits that are equal between the words $\mathbf{a}_m$ and $\mathbf{x}$. With this definition, a CAM declares a match if $\overline{h}(\mathbf{a}_m, \mathbf{x}) = N$. From a circuit perspective, the Hamming similarity can be computed by performing a *population count* that counts the number of ones over all XNOR outputs of the CAM bit-cells of a word.

In short, PPAC builds upon a CAM that is able to compute the Hamming similarity $\overline{h}(\mathbf{a}_m, \mathbf{x})$ for each word $\mathbf{a}_m$, $m = 1, \ldots, M$, in parallel during a single clock cycle. In addition, PPAC includes (i) an additional bit-cell operator (besides the XNOR) and (ii) a simple ALU per row that enables a wide range of applications. Since $\overline{h}(\mathbf{a}_m, \mathbf{x})$ is available, PPAC can implement not only a standard *complete-match* CAM that declares a match whenever $\overline{h}(\mathbf{a}_m, \mathbf{x}) = N$, but also a *similarity-match* CAM that declares a match whenever the number of equal bits between $\mathbf{a}_m$ and $\mathbf{x}$ meets a programmable threshold δ; i.e., $\overline{h}(\mathbf{a}_m, \mathbf{x}) \geq \delta$. As shown in Section III-A, this similarity-match functionality finds use in different applications.

It is important to realize that with the availability of the Hamming similarity $\overline{h}(\mathbf{a}_m, \mathbf{x})$, PPAC can also compute an inner-product between the vectors $\mathbf{a}_m$ and $\mathbf{x}$. Assume that the entries of the N-dimensional binary-valued vectors $\mathbf{a}_m$ and $\mathbf{x}$ are defined as follows: If the nth bit has a logical high (HI) value, then the nth entry represents a $+1$; if the nth bit has a logical low (LO) value, then the nth entry represents a -1. For this mapping, the inner-product between $\mathbf{a}$ and $\mathbf{x}$ is

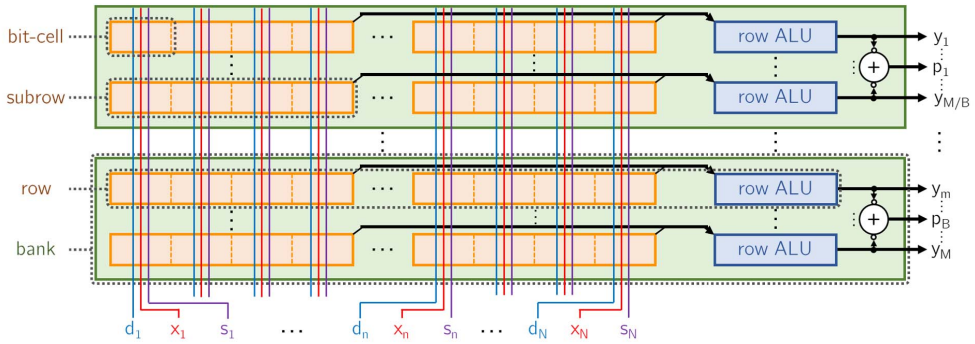
$$\langle \mathbf{a}_m, \mathbf{x} \rangle = \sum_{n=1}^{N} a_{m,n} x_n = 2\overline{h}(\mathbf{a}_m, \mathbf{x}) - N. \tag{1}$$

To see this, note that since $a_{m,n}, x_n \in \{\pm 1\}$, each of the partial products $a_{m,n}x_n$ is $+1$ if $a_{m,n} = x_n$ and -1 if $a_{m,n} \neq x_n$; this partial product can be computed with an XNOR. If all of the N entries between $\mathbf{a}_m$ and $\mathbf{x}$ differ, then $\langle \mathbf{a}_m, \mathbf{x} \rangle = -N$. Otherwise, for each bit n for which $a_{m,n} = x_n$, the partial product $a_{m,n}x_n$ will change from -1 to $+1$, increasing the inner-product sum by 2. As the total number of bits that are equal between $\mathbf{a}_m$ and $\mathbf{x}$ is given by $\overline{h}(\mathbf{a}_m, \mathbf{x})$, it follows that we can compute $\langle \mathbf{a}_m, \mathbf{x} \rangle$ as in (1). Note that PPAC computes the inner-product $\langle \mathbf{a}_m, \mathbf{x} \rangle$ in parallel for all the stored words $\mathbf{a}_m$, $m = 1, \ldots, M$, which is exactly a 1-bit MVP $\mathbf{Ax}$ between the matrix $\mathbf{A}$ (whose rows are the words $\mathbf{a}_m$) and the input vector $\mathbf{x}$. Such MVPs can be computed in a single clock cycle.

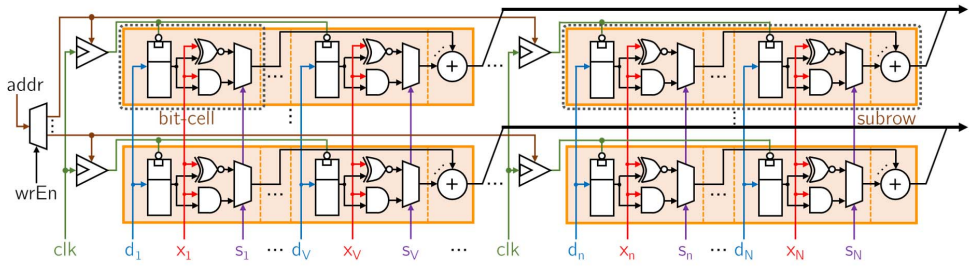
As we will show in Section III, PPAC can compute multi-bit MVPs *bit-serially* over several clock cycles. Furthermore, while the XNOR gate was used to multiply $\{\pm 1\}$ entries, an AND gate can be included in each bit-cell to enable the multiplication of $\{0, 1\}$ entries. With this AND functionality, PPAC can additionally perform (i) operations in GF(2), (ii) standard unsigned and 2's-complement signed arithmetic, and (iii) arbitrary Boolean functions in a similar fashion to a PLA.

B. Architecture Details

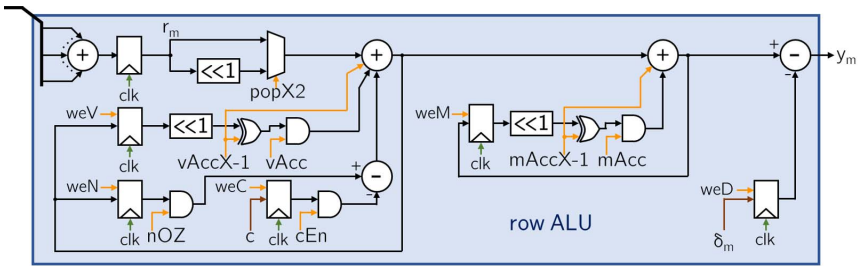
The high-level PPAC architecture is depicted in Fig. 2(a) and consists of multiple banks (green boxes) containing multiple



(a) High-level PPAC architecture.



(b) Bit-cell and subrow details.



(c) Row ALU details.

Fig. 2. Parallel Processor in Associative CAM (PPAC) architecture. (a) High-level architecture. (b) Each bit-cell includes an XNOR and an AND gate to perform bit-wise $\{\pm 1\}$ and $\{0, 1\}$ multiplications, respectively. Writing to the bit-cell latches is accomplished using clock gates. (c) Each row of bit-cells is connected to a row ALU; the fixed-amount shifters are used to multiply the input by two; control signals are colored in orange, external data inputs in brown.

rows. Each of the M PPAC rows stores an N-bit word in its memory (orange boxes) and is equipped with a row ALU (blue boxes). The row ALU adds the N one-bit results coming from all of the bit-cells on the row using a population count. The row population count is then used to perform different operations in the row ALU, such as Hamming-similarity or inner-product computation. Finally, each of the B banks (green boxes) contains a population count that sums up the negation of the most significant bits (MSBs) of all the row ALU's outputs. As detailed in Section III-E, this operation enables PPAC to implement PLA functionality.

The PPAC bit-cell architecture is depicted in Fig. 2(b). All of the bit-cells corresponding to the nth bit position in all words $m = 1, \ldots, M$ share three input signals: (i) $\mathtt{d}_n$ is the bit that will be stored in the bit-cell, (ii) $\mathtt{x}_n$ is the nth bit of the input word $\mathbf{x}$, and (iii) $\mathtt{s}_n$ determines if the bit-cell operator will be the XNOR or AND gate. Each bit-cell contains a memory element (an active-low latch) that stores the input $\mathtt{d}_n$. The bit-cells contain XNOR and AND gates to perform multiplications between the input $\mathtt{x}_n$ and the stored bit $a_{m,n}$, as well as a multiplexer, controlled by the input $\mathtt{s}_n$ that selects the bit-cell operation. The bit-cell storage elements are written only if the address `addr` corresponding to that row and the write enable signal `wrEn` are asserted; we use clock gates to implement this functionality. Once the memory elements are written and the control signal $\mathtt{s}_n$ has been fixed for each column, different input vectors $\mathbf{x}$ can be applied to PPAC. Then, the bit-cell operation results are passed to the row ALU, which accumulates the outputs and performs additional operations. To improve PPAC's scalability to large arrays, each row memory is divided into B_s subrows. Each subrow performs a population count over its $V = N/B_s$ bit-cells' results using a local adder. With this partitioning scheme, the number of wires between each subrow and the row ALU decreases from V to $\lceil \log_2(V+1) \rceil$,

where $\lceil \cdot \rceil$ is the ceiling function.

The PPAC row ALU architecture is depicted in Fig. 2(c). The row ALU first adds the incoming local population counts of all subrows and computes the total population count r_m of the bit-cells' results for the entire row m. Note that, when the XNOR operator is being used in all of the row's bit-cells, we have $r_m = \overline{h}(\mathbf{a}_m, \mathbf{x})$. The result r_m is then passed through two accumulators. The first accumulator is used in applications where the vector $\mathbf{x}$ has multi-bit entries. In this case, the MVP is carried out in a bit-serial fashion. The adder of the first accumulator also has an input to include an offset that can be used to adjust the row population count r_m according to the application. The second accumulator is used in applications where the matrix $\mathbf{A}$ has multi-bit entries. A programmable threshold δ_m is then subtracted from the output of the second accumulator to generate the row ALU's output y_m, whose interpretation depends on the operation mode. In Section III, we will describe how the row ALU is configured (and its output is interpreted) for each PPAC operation mode. Note that the row ALU contains two quantities that must be stored at configuration time: (i) The offset c used to correctly interpret the row population count r_m (the offset c is the same for all rows for a given application) and (ii) the threshold δ_m (the threshold δ_m can be different for each row). Finally, to increase the throughput of PPAC, we added a pipeline stage after the row population count; this increases the latency of all 1-bit operations to two clock cycles, but a new result of a 1-bit operation will be generated in every clock cycle.

III. PPAC Operation Modes and Applications

We now describe the different operating modes of the proposed PPAC and outline corresponding applications. In the following descriptions, we assume that all the unspecified control signals in the row ALU (cf. Fig. 2(c)) have a value of 0; write enable (`we`) signals are set as required by the operation.

A. Hamming Similarity

In this mode, PPAC computes the Hamming similarity between the M words $\mathbf{a}_m$, $m = 1, \ldots, M$, stored in each row and the input word $\mathbf{x}$. To this end, the bit-cells are configured to use the XNOR operator, so that the row population count r_m corresponds to $\overline{h}(\mathbf{a}_m, \mathbf{x})$. The row ALU is configured to pass this result to PPAC's output (by setting all control signals and δ_m to 0), so that $y_m = \overline{h}(\mathbf{a}_m, \mathbf{x})$ is the Hamming similarity.

By setting $\delta_m = N$, PPAC can be used as a regular CAM. If all the bits of the stored word $\mathbf{a}_m$ match the bits of $\mathbf{x}$, then $r_m = N$; hence, we have $y_m = 0$ and declare a match. Otherwise, if $r_m < N$, then $y_m < 0$. Thus, a complete-match can be declared by just looking at the MSB of the output y_m. By setting $0 \leq \delta_m \leq N$, PPAC declares a similarity-match whenever $\overline{h}(\mathbf{a}_m, \mathbf{x}) \geq \delta_m$. Note that PPAC performs M parallel Hamming-similarity computations in each clock cycle.

In this operation mode, PPAC can be used for applications that rely on CAMs [11], including network switches and routers [12], computer caches [13], and content-addressable parallel processors (CAPPs) [14], [15]. In this mode, PPAC can also be used for particle track reconstruction [7] and for locality-sensitive hashing (LSH), which enables computationally efficient approximate nearest neighbor search [16].

B. 1-bit Matrix-Vector-Products

In this mode, PPAC computes one MVP $\mathbf{y} = \mathbf{A}\mathbf{x}$ per clock cycle, where $y_m = \langle \mathbf{a}_m, \mathbf{x} \rangle$, $m = 1, \ldots, M$, and $\mathbf{a}_m$ and $\mathbf{x}$ are both N-dimensional vectors with 1-bit entries. We now detail how PPAC is able to support different 1-bit number formats.

1) Matrix and Vector with $\{\pm 1\}$ Entries: In this configuration, the LO and HI logical levels are interpreted as -1 and $+1$, respectively, for both the matrix $\mathbf{A}$ stored in PPAC and the input vector $\mathbf{x}$. Multiplication between a bit in $\mathbf{a}_m$ (the mth row of $\mathbf{A}$) and a bit in $\mathbf{x}$ can be computed via the bit-cell's XNOR gate. However, the row population count r_m is an unsigned number in the range $[0, N]$. To obtain the inner product $\langle \mathbf{a}_m, \mathbf{x} \rangle$ from r_m, we use (1), which can be implemented in the row ALU by setting `cEn` $= 1$, $c = N$, and `popX2` to double the row population count (by left-shifting r_m once).

2) Matrix and Vector with $\{0, 1\}$ Entries: In this configuration, the LO and HI logical levels are interpreted as 0 and 1, respectively, for both the matrix and input vector. Multiplication between a bit in $\mathbf{a}_m$ and a bit in $\mathbf{x}$ will be 1 only if both entries are 1; this corresponds to using the AND gate in each bit-cell. Hence, the row population count satisfies $r_m = \langle \mathbf{a}_m, \mathbf{x} \rangle$, which can be passed directly to the row ALU output y_m.

3) Matrix with $\{\pm 1\}$ and Vector with $\{0, 1\}$ Entries: In this configuration, the vector $\mathbf{x}$ is expressed as $\mathbf{x} = 0.5(\hat{\mathbf{x}} + \mathbf{1})$, where $\hat{\mathbf{x}}$ has $\{\pm 1\}$ entries and $\mathbf{1}$ is the all-ones vector. Note that $\hat{\mathbf{x}}$ can be easily obtained by setting the entries of $\mathbf{x}$ that are 0 to -1; i.e., $\hat{\mathbf{x}}$ and $\mathbf{x}$ are equivalent in terms of logical LO and HI levels. Using (1), we have the following equivalence:

$$\langle \mathbf{a}_m, \mathbf{x} \rangle = \overline{h}(\mathbf{a}_m, \hat{\mathbf{x}}) + \overline{h}(\mathbf{a}_m, \mathbf{1}) - N. \tag{2}$$

This requires us to compute $\overline{h}(\mathbf{a}_m, \mathbf{1})$, which can be obtained in the Hamming-similarity mode with input vector $\mathbf{1}$. The result of this operation is stored in the row ALU by setting `weN` to 1. To complete (2), the Hamming-similarity mode is applied again, but this time with $\mathbf{x}$ (which has the same logical representation as $\hat{\mathbf{x}}$) as the input vector, and with `nOZ` and `cEn` set to 1 and $c = N$. Note that $\overline{h}(\mathbf{a}_m, \mathbf{1})$ needs to be computed once only if the matrix $\mathbf{A}$ changes.

4) Matrix with $\{0, 1\}$ and Vector with $\{\pm 1\}$ Entries: In this configuration, the vector $\mathbf{x}$ is expressed as $\mathbf{x} = 2\tilde{\mathbf{x}} - \mathbf{1}$, where $\tilde{\mathbf{x}}$ has $\{0, 1\}$ entries and, as above, has the same logical LO and HI levels as $\mathbf{x}$. By noting that $\langle \mathbf{a}_m, \mathbf{1} \rangle = N - \overline{h}(\mathbf{a}_m, \mathbf{0})$, where $\mathbf{0}$ is the all-zeros vector, we have the following equivalence:

$$\langle \mathbf{a}_m, \mathbf{x} \rangle = 2\langle \mathbf{a}_m, \tilde{\mathbf{x}} \rangle + \overline{h}(\mathbf{a}_m, \mathbf{0}) - N. \tag{3}$$

As in (2), this requires us to compute $\overline{h}(\mathbf{a}_m, \mathbf{0})$, which can be obtained in the Hamming-similarity mode with input vector $\mathbf{0}$. The result of this operation is stored in the row ALU (by setting `weN` to 1). One can then compute a 1-bit $\{0, 1\}$ MVP to obtain $\langle \mathbf{a}_m, \tilde{\mathbf{x}} \rangle$ for all PPAC rows $m = 1, \ldots, M$, but this time with `popX2`, `nOZ`, and `cEn` set to 1, and $c = N$ to complete (3). As above, $\overline{h}(\mathbf{a}_m, \mathbf{0})$ has to be computed only if $\mathbf{A}$ changes.

1-bit $\{\pm 1\}$ MVPs can, for example, be used for inference of binarized neural networks [17]. While 1-bit MVPs in the other number formats might have limited applicability, they are used for multi-bit operations as described next.

C. Multi-bit Matrix-Vector-Products

In this mode, PPAC computes MVPs $\mathbf{y} = \mathbf{A}\mathbf{x}$ where the entries of $\mathbf{A}$ and/or $\mathbf{x}$ have multiple bits. All of these multi-bit operations are carried out in a bit-serial manner, which implies that MVPs are computed over multiple clock cycles.

1) Multi-bit Vector: Consider the case where $\mathbf{A}$ has 1-bit entries, while the vector $\mathbf{x}$ has L-bit entries. We start by writing

$$\mathbf{x} = \sum_{\ell=1}^{L} 2^{\ell-1}\mathbf{x}_\ell, \tag{4}$$

where $\mathbf{x}_\ell$ is a 1-bit vector formed by the ℓth bit of all the entries of $\mathbf{x}$. This decomposition enables us to rewrite the MVP as follows:

$$\mathbf{A}\mathbf{x} = \sum_{\ell=1}^{L} 2^{\ell-1}\mathbf{A}\mathbf{x}_\ell. \tag{5}$$

We use PPAC's 1-bit MVP mode with input $\mathbf{x}_L$ (the MSB of the entries of $\mathbf{x}$) to compute $\mathbf{A}\mathbf{x}_L$. The result is stored in the first accumulator of the row ALU by setting `weV` to 1. In the subsequent clock cycle, this value is doubled and added to $\mathbf{A}\mathbf{x}_{L-1}$ by setting `vAcc` to 1. By repeating this operation for $\ell = L, L-1, \ldots, 1$, the MVP $\mathbf{y} = \mathbf{A}\mathbf{x}$ is computed *bit-serially* in L clock cycles.

2) Multi-bit Matrix: Consider the case where each entry of $\mathbf{A}$ has K-bit entries. We use the same concept as in (5) and we decompose $\mathbf{A} = \sum_{k=1}^{K} 2^{k-1}\mathbf{A}_k$, where $\mathbf{A}_k$ is a 1-bit matrix formed by the kth bit of all entries of $\mathbf{A}$. In contrast to the multi-bit vector case, PPAC's memory cannot be replaced to contain a different matrix $\mathbf{A}_k$ every cycle. Instead, similar to [6], different columns of PPAC are used for different bit-significance levels, so that all K bits of the entries of $\mathbf{A}$ are stored in PPAC's memory. As a result, PPAC will now contain N/K different K-bit entries per row, instead of N different 1-bit entries per row. To ensure that only elements from $\mathbf{A}_k$ are used, the columns with different significance are configured to use the AND operator, and the corresponding entry of $\mathbf{x}$ is set to 0, effectively nulling any contribution from these columns to the row population count r_m. The rest of the columns are configured according to the used number format, and c in the row ALUs is set to N/K for the number formats that use it, so that PPAC computes $\mathbf{A}_k\mathbf{x}$ for an input $\mathbf{x}$ that has N/K entries of L bits. PPAC starts by computing $\mathbf{A}_K\mathbf{x}$ (i.e., the MVP using the most significant bit of the entries of $\mathbf{A}$) and saves the result in the second accumulator of the row ALU (by setting `weM` to 1), so that after L cycles (assuming each vector entry has L bits), it can double the accumulated result and add it to $\mathbf{A}_{K-1}\mathbf{x}$ by setting `mAcc` to 1. The new accumulated result is stored in the second accumulator, which will be written again L clock cycles later. By repeating this procedure, the multi-bit MVP $\mathbf{y} = \mathbf{A}\mathbf{x}$ is computed bit-serially over KL clock cycles.

TABLE I
L-BIT NUMBER FORMATS SUPPORTED BY PPAC

Name	`uint`	`int`	`oddint`
LO level	0	0	-1
HI level	1	1	1
Signed?	No	Yes	Yes
Min. value	0	-2^{L-1}	$-2^L + 1$
Max. value	$2^L - 1$	$2^{L-1} - 1$	$2^L - 1$
E.g., $L = 2$	$\{0, 1, 2, 3\}$	$\{-2, -1, 0, 1\}$	$\{-3, -1, 1, 3\}$

3) Supported Number Formats: As detailed in Section III-B, PPAC is able to compute multi-bit MVPs with different number formats summarized in Table I. For example, by mapping the logical LO level to 0 and HI to 1, multi-bit MVPs between unsigned numbers (`uint`) are performed. To operate with signed numbers (`int`), we negate (in 2's complement representation) the partial products $\mathbf{A}_k\mathbf{x}_L$ (for signed multi-bit vectors) or $\mathbf{A}_K\mathbf{x}$ (for signed multi-bit matrices), which are associated with the MSBs of the signed numbers in the vector $\mathbf{x}$ and matrix $\mathbf{A}$, respectively. We can configure the row ALUs to implement this behavior by setting `vAccX-1` and `mAccX-1` to 1 for a signed vector or matrix, respectively. The `oddint` number format arises from having a multi-bit number in which LO and HI get mapped to -1 and $+1$, respectively. Then, by applying (4), `oddint` represents signed odd numbers, as illustrated in Table I. Note that `oddint` cannot represent 0.

Low-resolution multi-bit MVPs using different number formats find widespread use in practice. For example, neural network inference can be executed with matrices and vectors using low-precision `int` numbers, where the threshold δ_m in the row ALU can be used as the bias term of a fully-connected (dense) layer. A 1-bit `oddint` matrix multiplied with a multi-bit `int` vector can be used to implement a Hadamard transform [18], which finds use in signal processing, imaging, and communication applications.

D. GF(2) Matrix-Vector-Products

In this mode, PPAC is able to perform MVPs in GF(2), the finite field with two elements $\{0, 1\}$. Multiplication in this field corresponds to an AND operation; addition corresponds to an XOR operation, which is equivalent to a simple addition modulo-2. GF(2) addition can then be performed by extracting the least significant bit (LSB) of a standard integer addition. To support MVPs in this mode, all of the columns of PPAC are set to use the AND operator in the bit-cells, and the row ALU is configured so that $y_m = r_m$. Then, the result of $\langle \mathbf{a}_m, \mathbf{x} \rangle$ in GF(2) can be extracted from the LSB of y_m. We emphasize that recent mixed-signal architectures that support MVPs, such as the ones in [6], [19], are unable to support this mode as the LSBs of analog additions are generally not bit-true.

GF(2) MVPs find widespread application in the computation of substitution boxes of encryption systems, including AES [20], as well as in encoding and decoding of error-correction codes, such as low-density parity-check [21] and polar codes [22].

E. Programmable Logic Array

In this mode, each PPAC bank is able to compute a Boolean function as a sum of min-terms, similar to a PLA. To this end, the mth row computes a min-term as follows: Each PPAC column and entry of the input vector $\mathbf{x}$ correspond to a different Boolean variable X; note that we consider the complement $\overline{X}$ as a different Boolean variable that is associated with another column and input entry. Then, if the Boolean variable associated with the nth column should appear in the min-term computed by the mth row, the $a_{m,n}$ bit-cell must store a logical 1, otherwise a logical 0. Furthermore, all PPAC columns are set to use the AND operator, and the row ALU is configured so that $y_m = r_m - \delta_m$, where the threshold δ_m must be the number of Boolean variables that are in the mth row's min-term (i.e., the number of logical 1's stored in $\mathbf{a}_m$). By doing so, $y_m = 0$ only if all of the Boolean variables in the min-term are 1; otherwise, $y_m < 0$. This implies that the result of the min-term of the mth PPAC row can be extracted from the complement of the MSB of y_m. Finally, the results of all min-terms in the bth bank are added together using the bank adder (see the adder in Fig. 2(a)). If $p_b > 0$, then at least one of the min-terms has a value of 1, so the output of the Boolean function programmed in the bank is a logical 1; otherwise, it is a logical 0.

Note that PPAC also supports different logic structures. For example, if we set $\delta_m = 1$, then each row will be computing a max-term. If we interpret the result of the Boolean function to be 1 only if p_b is equal to the number of programmed max-terms in the bank, PPAC effectively computes a product of max-terms. In general, PPAC can execute a logic function with two levels: The first stage can be a multi-operand AND, OR, or majority gate (MAJ) of the Boolean inputs; the second stage can be a multi-operand AND, OR, or MAJ of the outputs of the first stage. With this, PPAC can be used as a look-up table or programmed as a PLA that computes Boolean functions.

IV. Implementation Results

We now present post-layout implementation results of various PPAC array sizes in 28 nm CMOS and provide a comparison to existing in-memory accelerators and other related designs.

A. Post-Layout Implementation Results

We have implemented four different $M \times N$ PPAC arrays in 28 nm CMOS. All of these PPAC implementations have banks formed by 16 rows, each with $V = 16$ bit-cells per subrow, and a row ALU that supports multi-bit operations with L and K up to 4 bits. In Table II, we summarize our post-layout implementation results; the CAD-generated layout of the 256×256 PPAC design is shown in Fig. 3. The throughput is measured in operations (OP) per second, where we count both 1-bit multiplications and 1-bit additions as one OP each. Since each PPAC row performs an inner product between two N-dimensional 1-bit vectors, an $M \times N$ PPAC performs $M(2N–1)$ OP per clock cycle. Even if the clock frequency decreases as PPAC's dimensions increase, the overall throughput increases up to 92 TOP/s for the 256×256 array; this occurs due to the massive parallelism of our design. We also observe that

TABLE II
Post-layout implementation results for different PPAC array sizes in 28nm CMOS

Words M	16	16	256	256
Word-length N	16	256	16	256
Banks B	1	1	16	16
Subrows B_s	1	16	1	16
Area [μm^2]	14 161	72 590	185 283	783 240
Density [%]	75.77	70.45	72.52	72.13
Cell area [kGE]	17	81	213	897
Max. clock freq. [GHz]	1.116	0.979	0.824	0.703
Power [mW]	6.64	45.60	78.65	381.43
Peak throughput [TOP/s]	0.55	8.01	6.54	91.99
Energy-eff. [fJ/OP]	12.00	5.69	12.03	4.15

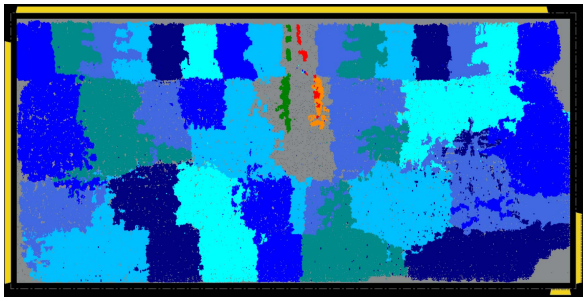


Fig. 3. Layout of the 256×256 PPAC with $B = B_s = 16$. All banks but one are colored using different shades of blue. For the gray bank, one row is shown in green, while the row memory and row ALU of another row are shown in orange and red, respectively.

increasing the number of words M results in a higher area and power consumption than increasing the number of bits per word N by the same factor. This behavior is due to the fact that adding a new row implies including a new row ALU, whose area can be comparable to that of the row memory (cf. Fig. 3). In contrast, increasing the number of bits per word N mainly modifies the datapath width of an existing row ALU, which scales only logarithmically in N, improving the energy-efficiency of the 256×256 PPAC to 4.15 fJ/OP.

In Table III, we summarize the throughput, power, and energy-efficiency for the different operation modes executed on a 256×256 PPAC. Throughput and energy-efficiency are measured in terms of MVPs, where for the Hamming-similarity mode, an MVP corresponds to the computation of $M = 256$ Hamming similarities; for the PLA mode, an MVP computes $B = 16$ distinct Boolean functions. To extract power estimates, we used Cadence Innovus and stimuli-based post-layout simulations at 0.9 V and 25 C° in the typical-typical process corner. In our simulations, we first load a randomly-generated matrix $\mathbf{A}$ into PPAC's memory, and then apply 100 random input vectors $\mathbf{x}$ for the 1-bit operations, while for the 4-bit $\{0, 1\}$ MVP case, we execute 100 different MVPs. We simulate the dynamic and static power consumption of PPAC only while performing computations (i.e., we exclude the power consumption of initializing the matrix $\mathbf{A}$), as this is the envisioned use case for PPAC—applications in which

TABLE III
THROUGHPUT, POWER, AND ENERGY-EFFICIENCY FOR DIFFERENT APPLICATIONS WITH A 256×256 PPAC ARRAY IN 28NM CMOS

Operation mode	Throughput [GMVP/s]	Power [mW]	Energy-eff. [pJ/MVP]
Hamming similarity	0.703	478	680
1-bit $\{\pm 1\}$ MVP	0.703	498	709
4-bit $\{0, 1\}$ MVP	0.044	226	5 137
GF(2) MVP	0.703	353	502
PLA	0.703	352	501

the matrix $\mathbf{A}$ remains largely static but the input vectors $\mathbf{x}$ change at a fast rate. From Table III, we observe that operations that use the XNOR operator (i.e., Hamming similarity and 1-bit $\{\pm 1\}$ MVP) exhibit higher power consumption than tasks relying on the AND operation; this is because the switching activity at the output of XNOR gates is, in general, higher than that of AND gates.

B. Comparison with Existing Accelerators

In Table IV, we compare the 256×256 PPAC with existing hardware accelerators that have been specialized for binarized neural network (BNN) inference and support fully-connected layers [6], [10], [19], [23], [24]. We compare against these designs as their operation closely resembles that of PPAC's 1-bit $\{\pm 1\}$ MVP operation mode. In fact, all of the considered designs count 1-bit products and additions as one operation (OP) each—an inner product between two N-dimensional 1-bit vectors is $2N$ OPs. The designs in [6], [10] are PIM accelerators in which part of the computation is carried out within the bit-cells; the designs in [6], [19] rely on mixed-signal techniques to compute MVPs.

By considering technology scaling, we see that the energy efficiency (in terms of TOP/s/W) of PPAC is comparable to that of the two fully-digital designs in [23], [24] but $7.9\times$ and $2.3\times$ lower than that of the mixed-signal designs in [6] and [19], respectively, where the latter is implemented in a comparable technology node as PPAC. As noted in Section III-D, mixed-signal designs are particularly useful for tasks that are resilient to noise or process variation, such as neural network inference. However, mixed-signal designs cause issues in applications that require bit-true results, such as addition in GF(2), which requires the LSB of an integer addition to be exact.

We also see that PPAC achieves the highest peak throughput among the considered designs, which is due to its massive parallelism. We emphasize, however, that PPAC's performance was extracted from post-layout simulations, whereas all the other designs, except that in [24], are silicon-proven. Furthermore, all other designs not only execute 1-bit MVPs, but they also include other operations that are required to implement BNN inference, such as activation functions and batch normalization. PPAC, in contrast, is unable to completely execute BNN inference, but is able to execute a 256×256 MVP followed by adding a bias vector, which is a large portion of the operations required to process a fully-connected BNN layer. As a result, the reported throughput and energy-efficiency for PPAC are optimistic.

We would like to reiterate that PPAC is a massively-parallel PIM engine that can be used for a number of different MVP-like operations, where 1-bit MVP is just one of them. As such, the main purpose of the comparison in Table IV is to demonstrate that PPAC's 1-bit $\{\pm 1\}$ MVP operation mode holds promise with an energy-efficiency that is comparable to that of other accelerators. While the hardware designs in [10], [19], [24] are specialized to carry out 1-bit MVPs and the designs in [6], [23] to execute multi-bit MVPs for neural network inference, PPAC is programmable to perform not only these operations, but also GF(2) MVPs, Hamming-similarity computations, and PLA or CAM functionality, opening up its use in a wide range of applications. In this sense, PPAC is similar to the work in [3], where PIM is used to accelerate multiple applications, such as database query processing, cryptographic kernels, and in-memory checkpointing. A fair comparison to [3] is, however, difficult as it considers a complete system—PPAC would need to be integrated into a system for a fair comparison. We note, however, that if the method in [3] is used to compute MVPs, an element-wise multiplication between two vectors whose entries are L-bit requires $L^2 + 5L - 2$ clock cycles [4], which is a total of 34 clock cycles for 4-bit numbers. Then, the reduction (via sum) of an N-dimensional vector with L-bits per entry requires $\mathcal{O}(L \log_2(N))$ clock cycles, which is at least 64 clock cycles for a 256-dimensional vector with 8-bit entries (as the product of two 4-bit numbers results in 8-bit). Hence, an inner product between two 4-bit vectors with 256 entries requires at least 98 clock cycles—PPAC requires only 16 clock cycles for the same operation. This significant difference in the number of clock cycles is caused by the fact that the design in [4] is geared towards data-centric applications in which element-wise operations are performed between high-dimensional vectors to increase parallelism. PPAC aims at accelerating a wide range of MVP-like operations, which is why we included dedicated hardware (such as the row pop-count) to speed up element-wise vector multiplication and vector sum-reduction.

V. CONCLUSIONS

We have developed a novel, all-digital in-memory accelerator we call *Parallel Processor in Associative CAM* (PPAC). PPAC accelerates a variety of matrix-vector-product-like operations with different number formats in a massively-parallel manner. We have provided post-layout implementation results in a 28nm CMOS technology for four different array sizes, which demonstrate that a 256×256 PPAC array achieves 92 TOP/s at an energy efficiency of 4.15 fJ/OP. Our comparison with recent digital and mixed-signal PIM and non-PIM accelerators has revealed that PPAC can be competitive in terms of throughput and energy-efficiency while maintaining high flexibility.

We emphasize that the all-digital nature of PPAC has numerous practical advantages over existing mixed-signal PIM designs. First, PPAC can be implemented using automated CAD tools with conventional standard-cell libraries and fabricated in standard CMOS technologies. Second, PPAC is written in

TABLE IV
COMPARISON WITH EXISTING BINARIZED NEURAL NETWORK (BNN) ACCELERATOR DESIGNS

Design	PIM?	Mixed signal?	Implementation	Technology [nm]	Supply [V]	Area [mm^2]	Peak TP [GOP/s]	Energy-eff. [TOP/s/W]	Peak TP[a] [GOP/s]	Energy-eff.[a] [TOP/s/W]
PPAC	yes	no	layout	28	0.9	0.78	91 994	184	91 994	184
CIMA [6]	yes	yes	silicon	65	1.2	8.56	4 720	152	10 957	1 456
Bankman *et al.* [19]	no	yes	silicon	28	0.8	5.95	–	532	–	420
BRein [10]	yes	no	silicon	65	1.0	3.9	1.38	2.3	3.2	15
UNPU [23]	no	no	silicon	65	1.1	16	7 372	46.7[b]	17 114	376
XNE [24]	no	no	layout	22	0.8	0.016	108	112	84.7	54.6

[a]Technology scaling to 28 nm CMOS at $V_{\text{dd}} = 0.9$ V assuming standard scaling rules $A \sim 1/\ell^2$, $t_{\text{pd}} \sim 1/\ell$, and $P_{\text{dyn}} \sim 1/(V_\ell^2 \ell)$.
[b]Number reported in [23, Fig. 13]; note that the peak TP (7 372 GOP/s) divided by the reported power consumption (297 mW) yields 24.8 TOP/s/W.

RTL with Verilog, is highly parametrizable (in terms of array size, banking, supported operation modes, etc.), and can easily be migrated to other technology nodes. Third, PPAC's all-digital nature renders it robust to process variations and noise, facilitates in-silicon testing, and its clock frequency and supply voltage can be aggressively scaled to either increase throughput or improve energy-efficiency.

There are numerous avenues for future work. The design of semi-custom bit-cells (e.g., by fusing latches with logic) has the potential to significantly reduce area and power consumption, possibly closing the efficiency gap to mixed-signal PIM accelerators. Furthermore, guided cell placement and routing may yield higher bit-cell density and hence, potentially reduce area as well as mitigate interconnect congestions and energy. Finally, integrating PPAC into a processor either as an accelerator or compute cache is an interesting open research direction.

REFERENCES

[1] W. Wulf and S. McKee, "Hitting the memory wall: Implications of the obvious," *ACM SIGARCH Computer Architecture News*, vol. 23, no. 1, pp. 20–24, March 1995.

[2] R. Nair, "Evolution of memory architecture," *Proceedings of the IEEE*, vol. 103, no. 8, pp. 1331–1345, August 2015.

[3] S. Aga, S. Jeloka, A. Subramaniyan, S. Narayanasamy, D. Blaauw, and R. Das, "Compute caches," in *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, February 2017, pp. 481–492.

[4] C. Eckert, X. Wang, J. Wang, A. Subramaniyan, R. Iyer, D. Sylvester, D. Blaauw, and R. Das, "Neural cache: Bit-serial in-cache acceleration of deep neural networks," in *Proceedings of the ACM/IEEE International Symposium on Computer Architecture (ISCA)*, June 2018, pp. 383–396.

[5] Q. Guo, X. Guo, R. Patel, E. İpek, and E. Friedman, "AC-DIMM: Associative computing with STT-MRAM," in *Proceedings of the ACM/IEEE International Symposium on Computer Architecture (ISCA)*, June 2013, pp. 189–200.

[6] H. Jia, Y. Tang, H. Valavi, J. Zhang, and N. Verma, "A microprocessor implemented in 65nm CMOS with configurable and bit-scalable accelerator for programmable in-memory computing," *arXiv preprint: 1811.04047*, pp. 1–10, November 2018. [Online]. Available: https://arxiv.org/abs/1811.04047

[7] A. Annovi, G. Calderini, S. Capra, B. Checcucci, F. Crescioli, F. De Canio, G. Fedi, L. Frontini, M. Garci, C. Gentsos, T. Kubota, V. Liberali, F. Palla, J. Shojaii, C.-L. Sotiropoulou, A. Stabile, G. Traversi, and S. Viret, "Characterization of an associative memory chip in 28 nm CMOS technology," in *Proceedings of the IEEE International Symposium in Circuits and Systems (ISCAS)*, May 2018, pp. 1–4.

[8] D. Kim, J. Kung, S. Chai, S. Yalamanchili, and S. Mukhopadhyay, "Neurocube: A programmable digital neuromorphic architecture with high-density 3D memory," in *Proceedings of the ACM/IEEE International Symposium on Computer Architecture (ISCA)*, June 2016, pp. 380–392.

[9] S. Li, D. Niu, K. Malladi, H. Zheng, B. Brennan, and Y. Xie, "DRISA: A DRAM-based reconfigurable in-situ accelerator," in *Proceedings of the IEEE/ACM International Symposium on Microarchitecture (MICRO)*, October 2017, pp. 288–301.

[10] K. Ando, K. Ueyoshi, K. Orimo, H. Yonekawa, S. Sato, H. Nakahara, S. Takameaeda-Yamazaki, M. Ikebe, T. Asai, T. Kuroda, and M. Motomura, "BRein memory: A single-chip binary/ternary reconfigurable in-memory deep neural network accelerator achieving 1.4 TOPS at 0.6 W," *IEEE Journal Of Solid-State Circuits (JSSC)*, vol. 53, no. 4, pp. 983–994, April 2018.

[11] K. Pagiamtzis and A. Sheikholeslami, "Content-addresable memory (CAM) circuits and architectures: A tutorial and survey," *IEEE Journal Of Solid-State Circuits (JSSC)*, vol. 41, no. 3, pp. 712–727, March 2006.

[12] T.-B. Pei and C. Zukowski, "VLSI implementation of routing tables: tries and CAMs," in *Proceedings of the IEEE Conference on Computer Communications (INFCOM)*, April 1991, pp. 515–524.

[13] M. Zhang and K. Asanović, "Highly-associative caches for low-power processors," in *Kool Chips Workshop, IEEE/ACM International Symposium on Microarchitecture (MICRO)*, December 2000, pp. 1–6.

[14] C. Foster, *Content Addressable Parallel Processors*. John Wiley and Sons, Inc., 1976.

[15] C. Stormon, N. Troullinos, E. Saleh, A. Chavan, M. Brule, and J. Oldfield, "A general-purpose CMOS associative processor IC and system," *IEEE Micro*, vol. 12, no. 6, pp. 68–78, December 1992.

[16] A. Andoni and P. Indyk, "Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions," *Communications of the ACM*, vol. 51, no. 1, pp. 117–122, January 2008.

[17] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, December 2016, pp. 4114–4122.

[18] T. Goldstein, L. Xu, K. F. Kelly, and R. Baraniuk, "The STOne transform: Multi-resolution image enhancement and compressive video," *IEEE Transactions on Image Processing*, vol. 24, no. 12, pp. 5581–5593, December 2015.

[19] D. Bankman, L. Yang, B. Moons, M. Verhelst, and B. Murmann, "An always-on 3.8μJ/86% CIFAR-10 mixed-signal binary CNN processor with all memory on chip in 28nm CMOS," in *IEEE International Solid-State Circuits Conference (ISSCC)*, February 2018, pp. 222–224.

[20] J. Daemen and V. Rijmen, *The design of Rijndael: AES - The Advanced Encryption Standard*. Springer Science & Business Media, 2002.

[21] K. Cushon, P. Larsson-Edefors, and P. Andrekson, "A high-throughput low-power soft bit-flipping LDPC decoder in 28 nm FD-SOI," in *Proceedings of the IEEE European Solid State Circuits Conference (ESSCIRC)*, September 2018, pp. 102–105.

[22] E. Arıkan, "Channel polarization: A method for constructing capacity-achieving codes," in *IEEE International Symposium on Information Theory (ISIT)*, July 2008, pp. 1173–1177.

[23] J. Lee, C. Kin, S. Kang, D. Shin, S. Kim, and H.-J. Yoo, "UNPU: An energy-efficient deep neural network accelerator with fully variable weight bit precision," *IEEE Journal of Solid-State Circuits (JSSC)*, vol. 54, no. 1, pp. 173–185, January 2019.

[24] F. Conti, P. D. Schiavone, and L. Benini, "XNOR neural engine: A hardware accelerator IP for 21.6-fJ/op binary neural network inference," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 11, pp. 2940–2951, November 2018.